DBT PORTFOLIO PROJECT

LAYER: INTERMEDIATE

DBT Intermediate Model

Multiple Regression | OLS | Cramer's Rule | Statistics | Snowflake

Project Summary

Implements two-predictor Ordinary Least Squares regression entirely in SQL using Cramer's Rule to solve the 2x2 normal equation system. Mean-centers predictors for numerical stability, computes both slope coefficients and the intercept analytically, and outputs fitted values and residuals per entity/date.

Thomas Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Model Name:	int_multiple_linear_regression
Layer:	Intermediate
Materialization:	view
Predictors:	x1_value (e.g., claim volume), x2_value (e.g., denial rate)
Key Outputs:	beta_0, beta_1, beta_2, fitted_value, residual per entity/date
Audience:	Finance, Actuarial teams, Data Science

2 BUSINESS PROBLEM

Simple linear regression uses only one predictor -- but claim metrics are driven by multiple factors simultaneously (e.g., both claim volume AND denial rate together predict total reimbursement). This model extends OLS to two predictors, letting analysts quantify the independent effect of each driver while controlling for the other -- all without leaving SQL.

- Quantifies how much each predictor (x1, x2) independently drives the outcome (y).
- Mean-centering eliminates collinearity sensitivity in the normal equation solver.
- Cramer's Rule solves the 2x2 system algebraically -- no matrix libraries needed.
- Residuals flag entities where the two-predictor model still under- or over-fits.

3 TECHNICAL EXPLANATION

- source_data: Aliases y_value, x1_value, x2_value for clarity.
- stats: Aggregates per entity: n, averages, sum of squares and cross-products needed for normal equations.
- centered: Subtracts entity means from y, x1, x2 -- working in mean-centered space simplifies the 2x2 system.
- centered_stats: Computes sxx1, sxx2, sx1x2, sx1y, sx2y -- the five sufficient statistics for 2-predictor OLS.
- params: Applies Cramer's Rule: beta_1 = (sx1y*sxx2 - sx2y*sx1x2) / (sxx1*sxx2 - sx1x2^2). NULLIF guards determinant=0.
- params_with_intercept: beta_0 = y_bar - beta_1*x1_bar - beta_2*x2_bar.
- fitted: Joins source data to params; computes fitted = beta_0 + beta_1*x1 + beta_2*x2 and residual.

Key Techniques: Cramer's Rule, mean-centering, grouped aggregation, NULLIF determinant guard, multi-CTE join chain

4 FULL MODEL CODE

```
{{ config(materialized='view') }}

with source_data as (
    select entity_id, period_date,
           y_value as y, x1_value as x1, x2_value as x2
    from {{ source('healthcare', 'claims') }}
),
stats as (
    select entity_id, count(*) as n,
           avg(y) as y_bar, avg(x1) as x1_bar, avg(x2) as x2_bar,
           sum(x1*x1) as sum_x1sq, sum(x2*x2) as sum_x2sq,
           sum(x1*x2) as sum_x1x2,
           sum(x1*y) as sum_x1y,  sum(x2*y) as sum_x2y,
           sum(y*y)  as sum_ysq
    from source_data group by entity_id
),
centered as (
    select s.entity_id, s.period_date,
           s.y - st.y_bar   as yc,
           s.x1 - st.x1_bar as x1c,
           s.x2 - st.x2_bar as x2c,
           st.y_bar, st.x1_bar, st.x2_bar
    from source_data s join stats st using (entity_id)
),
centered_stats as (
    select entity_id,
           sum(x1c*x1c) as sxx1, sum(x2c*x2c) as sxx2,
           sum(x1c*x2c) as sx1x2,
           sum(x1c*yc)  as sx1y,  sum(x2c*yc)  as sx2y,
           max(y_bar) as y_bar, max(x1_bar) as x1_bar, max(x2_bar) as x2_bar
    from centered group by entity_id
),
params as (
    select entity_id,
           (sx1y*sxx2 - sx2y*sx1x2) / nullif((sxx1*sxx2 - sx1x2*sx1x2),0) as beta_1,
           (sx2y*sxx1 - sx1y*sx1x2) / nullif((sxx1*sxx2 - sx1x2*sx1x2),0) as beta_2,
           y_bar, x1_bar, x2_bar
    from centered_stats
),
params_with_intercept as (
    select *, y_bar - beta_1*x1_bar - beta_2*x2_bar as beta_0
    from params
),
fitted as (
    select s.entity_id, s.period_date, s.y as actual_value,
           s.x1, s.x2, p.beta_0, p.beta_1, p.beta_2,
           p.beta_0 + p.beta_1*s.x1 + p.beta_2*s.x2 as fitted_value,
           s.y - (p.beta_0 + p.beta_1*s.x1 + p.beta_2*s.x2) as residual
    from source_data s join params_with_intercept p using (entity_id)
)
select * from fitted order by entity_id, period_date
```